MANUAL TESTING REFERENCE

Swagger UI Testing Guide

Construction Site AI API — written for someone with zero FastAPI/Swagger experience. Follow this top to bottom, in order — later steps depend on IDs and tokens from earlier ones.

Every ID and expected output below is real, captured from this project's actual running server.

Before You Start

1. Start the server

Open a terminal (PowerShell), go to the project folder, and run:

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Wait until you see:

```
INFO:     Application startup complete.
```

> **Leave this window open and running.** Don't close it, don't press Ctrl+C, don't type anything else in it. If you need to run more commands, open a **second** terminal window.

2. Open Swagger UI

In your browser, go to: **http://127.0.0.1:8000/docs**

This is a list of all endpoints grouped by category (Health, Auth, Audio, Daily Logs, Projects).

3. How "Try it out" works (same for every endpoint)

1. Click the endpoint's blue/green bar to expand it
2. Click the **"Try it out"** button (top right of the expanded section)
3. Fill in any required fields
4. Click the blue **"Execute"** button
5. Scroll down to **"Server response"** → **"Response body"** to see what came back

---

`PART 1 — HEALTH ENDPOINTS (NO LOGIN NEEDED)`

These have no lock icon — you can test them immediately, before logging in.

`GET` /api/v1/health/live

What it checks: is the server process even running? No database, no external calls — the fastest, simplest check.

`EXPECTED — CODE 200`

```
{
  "success": true,
  "message": "Process is alive.",
  "data": { "status": "alive", "uptime_seconds": 12.3 }, ...
}
```

If you get this, the server is up. If you get a connection error instead, your `uvicorn` terminal isn't running.

`GET` /api/v1/health/ready

What it checks: can the server actually reach the PostgreSQL database?

`EXPECTED — CODE 200`

```
{ "success": true, "message": "Ready.", "data": { "status": "ready", "database": true } }
```

If `"database": false`, PostgreSQL isn't running or your `.env` has the wrong connection details.

`GET` /api/v1/health/version

What it checks: what version/environment is running. No I/O at all.

`EXPECTED — CODE 200`

```
{ "data": { "app_version": "0.7.0", "environment": "development", "api_version": "v1" } }
```

`GET` /api/v1/health (the full one)

What it checks: database **and** the Groq AI engine — the "everything OK?" check. Takes ~1 second longer since it makes a real call to Groq.

`EXPECTED — CODE 200`

```
{ "data": { "status": "up", "components": {
    "database": { "status": "up" },
    "groq_extraction_engine": { "status": "up" } } } }
```

If `groq_extraction_engine` shows `"down"`, check your `.env` has a valid `GROQ_API_KEY`.

`PART 2 — AUTHENTICATION`

You now need to log in. Every endpoint below this point has a lock icon — it needs a token.

`POST` /api/v1/auth/login

STEPS

1. Click `POST /api/v1/auth/login` under "Auth"
2. Try it out
3. Replace the request body with exactly:

```
{ "email": "admin@example.com", "password": "Admin@123" }
```

4. Execute

EXPECTED — CODE 200

```
{
  "success": true, "message": "Login successful.",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...(long string)...",
    "token_type": "bearer", "expires_in_minutes": 60,
    "role": "owner", "email": "admin@example.com"
  }
}
```

Copy the entire `access_token` value — you need it for the next step.

> This is the one working demo login built into the project. `admin@example.com` / `Admin@123` is intentional test data.

Now test the failure case too: body `{"email": "admin@example.com", "password": "wrongpassword"}` → **Expected: Code 401**, `{"success": false, "message": "Incorrect email or password."}`

Authorize Swagger with your token

This unlocks every other endpoint — do this once, then it applies for the rest of your session.

1. Scroll to the **top** of the Swagger page
2. Click the green **"Authorize"** button (top right)
3. A popup appears with a field labeled `HTTPBearer (http, Bearer)`
4. Paste your `access_token` value in — **just the raw token, no "Bearer" prefix**, Swagger adds that automatically
5. Click **Authorize**, then **Close**

PART 3 — DAILY LOGS (CRUD + REVIEW LIFECYCLE)

Use this real, pre-existing daily log ID for everything in this section:

```
aaaaaaaa-0008-4000-8000-000000000008
```

GET /api/v1/daily-logs/{log_id}

STEPS

1. Click `GET /api/v1/daily-logs/{log_id}` under "Daily Logs"
2. Try it out
3. In the `log_id` field, paste: `aaaaaaaa-0008-4000-8000-000000000008`
4. Execute

EXPECTED — CODE 200


```
{
  "success": true, "data": {
    "id": "aaaaaaaa-0008-4000-8000-000000000008",
    "current_stage": "framing", "review_status": "approved",
    "total_workers_present": 7,
    "trades_on_site": [ {...}, {...} ],
    "work_items": [ {...}, {...}, {...} ],
    "hazards": [ {...} ], ...many more fields...
  }
}
```


This is a full construction daily log with all its nested detail — trades on site, work completed, materials used, safety hazards, etc.

Now test the error cases:

- Fake ID `00000000-0000-0000-0000-000000000000` → **Code 404**, `{"success": false, "message": "Daily log not found."}`
- Garbage text `not-a-real-id` → **Code 422** (validation error, not a valid UUID shape)

The review lifecycle is a strict state machine — read before testing

The three endpoints below (`submit`, `approve`, `reject`) only allow specific transitions. Trying an invalid one **correctly fails with 409 Conflict** — that's the state machine protecting the workflow, not a bug:

ACTION	ALLOWED FROM STATUS	RESULT
submit	draft	→ under_review
approve	draft or under_review	→ approved
reject	under_review	→ rejected

Key rule: `approve` does NOT accept a log already `approved` or `rejected`. Calling it again correctly returns 409 — this is expected, not something to "fix." The seeded log (`aaaaaaaa-0008-...`) starts `approved`, so testing `submit` / `approve` directly on it will 409 immediately — check its status first (below).

Check the log's current status before testing

1. `GET /api/v1/daily-logs/{log_id}` → Execute → look at `review_status`
2. If `"draft"` → go to Submit below.
3. If `"under_review"` → go to Approve below.
4. If `"approved"` / `"rejected"` → needs a manual reset (there's no "unapprove" endpoint, by design — approvals stay auditable).

POST /api/v1/daily-logs/{log_id}/submit

What it does: moves a log from `draft` to `under_review`. Only works if currently `draft`.

EXPECTED — CODE 200

```
{ "success": true, "message": "Submitted for review.",
  "data": { "review_status": "under_review", "reviewed_by_id": null, ... } }
```

If NOT currently draft — Code 409 (correct behavior):

```
{ "success": false, "message": "Cannot submit log ...: current status is 'approved' (must be
'draft')" }
```

POST /api/v1/daily-logs/{log_id}/approve

What it does: moves a log from `draft` or `under_review` to `approved`.

REQUEST BODY

```
{ "notes": "Looks good, approving." }
```

EXPECTED — CODE 200

```
{ "success": true, "message": "Log approved.",
  "data": { "review_status": "approved",
    "review_notes": "Looks good, approving.",
    "reviewed_by_id": "aaaaaaaa-0009-...", "reviewed_at": "2026-07-14T15:28:28Z", ... } }
```

Note: only `owner` and `project_manager` roles can approve — a `foreman`-role token gets `403 Forbidden`.

Calling it again (now already approved) → correctly **Code 409**. Approve is a one-way, one-time transition per review cycle.

POST /api/v1/daily-logs/{log_id}/reject

What it does: moves a log from `under_review` to `rejected`, with required notes.

REQUEST BODY — NOTES IS REQUIRED

```
{ "notes": "Missing safety documentation, please redo." }
```

EXPECTED — CODE 200

```
{ "data": { "review_status": "rejected", "review_notes": "Missing safety documentation, please redo." } }
```

Test the validation: empty body `{}` → **Code 422** (notes required).

POST /api/v1/daily-logs/{log_id}/generate

What it does: runs the real Groq AI to generate all 4 business documents from this log's data. Takes **10-40 seconds** — don't panic if it looks stuck.

EXPECTED — CODE 200 (AFTER THE WAIT)

```
{ "success": true, "message": "Generated 4 document(s).",
  "data": { "outputs_generated": 4,
    "service_types": ["daily_report", "customer_update", "safety_talk", "material_reminder"] } }
```

GET /api/v1/daily-logs/{log_id}/outputs

What it does: shows every AI-generated document ever created for this log — a full history, not just the latest.

EXPECTED — CODE 200

```
{ "message": "Found 4 output(s).",
  "data": [
    { "service_type": "daily_report", "content": "## Daily Site Report...(real text)..." },
    { "service_type": "customer_update", "content": "Subject: Project Update...(real text)..." },
    { "service_type": "safety_talk", "content": "## Toolbox Talk...(real text)..." },
    { "service_type": "material_reminder", "content": "## Material Reminder...(real text)..." }
  ] }
```

Ran /generate more than once? Each run adds 4 *new* documents — it never overwrites previous ones. Run it 5 times and you'll correctly see `"Found 20 output(s)."` (5 × 4). A count that's a multiple of 4 means it's working correctly, not broken — it's a full audit history by design.

PART 4 — PROJECTS

GET /api/v1/projects/{project_id}/daily-logs

Use this real project ID:

```
aaaaaaaa-0006-4000-8000-000000000006
```

EXPECTED — CODE 200

```
{ "success": true, "message": "Found 1 log(s).",
  "data": [ { "id": "aaaaaaaa-0008-...", "review_status": "approved", ... } ],
  "metadata": { "total": 1, "limit": 30, "offset": 0, "count": 1 } }
```

Try the filter: `status=draft` → Execute → expect empty list (this log isn't a draft).

> **Don't confuse the Project ID with a Daily Log ID.** A Project is the long-running construction job (one company job that lasts weeks/months); a Daily Log is one single day's entry within it. One project has *many* daily logs, so each needs its own separate ID. The URL path tells you which one a field wants: `/projects/{project_id}/...` vs `/daily-logs/{log_id}/...`.

`PART 5 — AUDIO UPLOAD (THE FULL AI PIPELINE)`

A real `.wav` file goes in, and Whisper transcription → Groq extraction → database save → Groq document generation → database save all happen automatically in the background.

`POST` /api/v1/audio/upload

STEPS

1. Click `POST /api/v1/audio/upload` under "Audio"
2. Try it out
3. `file` field → Choose File → select `data\sample_audio\foreman_recording.wav`
4. `project_id` field → clear the placeholder and type the real project ID:

   ```
   aaaaaaaa-0006-4000-8000-000000000006
   ```

5. Execute

EXPECTED — CODE 202

```
{ "success": true, "message": "Upload accepted. Processing has started.",
  "data": { "id": "SOME-NEW-UUID", "original_filename": "foreman_recording.wav",
    "processing_status": "pending" } }
```

Copy the `id` value — needed for the next step.

> **Important:** Swagger pre-fills `project_id` with a fake example UUID like `3fa85f64-5717-4562-b3fc-2c963f66afa6`. That value does not exist in the database — always replace it with a real ID (like the one above) before executing, or you'll get a raw error instead of a clean response.

`GET` /api/v1/audio/{audio_file_id}/status

This field wants the **UUID from the upload response**, not a filename.

FIRST RESPONSE — CODE 200

```
{ "data": { "processing_status": "transcribing", "daily_log_id": null } }
```

Keep clicking Execute every few seconds (Swagger doesn't auto-refresh). Status progresses:

```
pending → transcribing → extracting → generating → complete
```

Takes **20-45 seconds total** — real Whisper (local) and Groq (cloud) calls.

FINAL RESPONSE — CODE 200

```
{ "data": { "processing_status": "complete", "daily_log_id": "SOME-NEW-UUID", "error_message": null
} }
```

Copy that `daily_log_id` — plug it into Part 3's `GET /daily-logs/{log_id}` and `/outputs` to see your own recording's results.

What if you upload the same day's recording twice?

Same project, same calendar day → a clean rejection, not a crash:

```
{ "error_message": "A daily log already exists for this project on 2026-07-14
  (daily_log_id=...). Upload rejected to avoid overwriting it." }
```

This is intentional — one log per project per day, with a clear pointer to the log that's blocking you.

Quick Reference — All Real IDs

WHAT	ID
Company	`aaaaaaaa-0001-4000-8000-000000000001`
Project	`aaaaaaaa-0006-4000-8000-000000000006`
Seeded daily log	`aaaaaaaa-0008-4000-8000-000000000008`
Login email	`admin@example.com`
Login password	`Admin@123`

Never use Swagger's greyed-out placeholder text as a real value — anywhere you see `3fa85f64-5717-4562-b3fc-2c963f66afa6` or similar, that's just an example format. Always replace it with a real ID from the table above, or one from a previous response.

Full Test Checklist

- ☐ 1.1 GET /health/live → 200
- ☐ 1.2 GET /health/ready → 200, database: true
- ☐ 1.3 GET /health/version → 200
- ☐ 1.4 GET /health → 200, both components up
- ☐ 2.1 POST /auth/login (correct password) → 200, copy token
- ☐ 2.1b POST /auth/login (wrong password) → 401
- ☐ 2.2 Click Authorize, paste token
- ☐ 3.1 GET /daily-logs/{id} (real id) → 200, full nested data
- ☐ 3.1b GET /daily-logs/{id} (fake id) → 404
- ☐ 3.1c GET /daily-logs/{id} (garbage id) → 422

- [ ] 3.2a Check review_status first — confirm draft before continuing
- [ ] 3.2b POST /daily-logs/{id}/submit (draft) → 200, under_review
- [ ] 3.2c POST /daily-logs/{id}/submit again → 409 (expected)
- [ ] 3.3 POST /daily-logs/{id}/approve (under_review) → 200, approved
- [ ] 3.3b POST /daily-logs/{id}/approve again → 409 (expected)
- [ ] 3.4 POST /daily-logs/{id}/reject (under_review) → 200, rejected
- [ ] 3.4b POST /daily-logs/{id}/reject (empty body) → 422
- [ ] 3.5 POST /daily-logs/{id}/generate → 200 (wait ~10-40s)
- [ ] 3.6 GET /daily-logs/{id}/outputs → 200, 4 real documents
- [ ] 4.1 GET /projects/{id}/daily-logs → 200, 1 log
- [ ] 5.1 POST /audio/upload (real project_id, real file) → 202
- [ ] 5.2 GET /audio/{id}/status (poll until complete) → 200, complete
- [ ] 5.3 (optional) upload same file again same day → clean rejection

If every box passes with the expected result, the entire API surface is confirmed working end to end — real database, real AI, real audio pipeline.
